

Report: Budgeted Maximum Weight Clique using MPI

COL7880: Assignment 3

Dipesh Kant (2025MCS2110)

April 10, 2026

1 Parallelization Strategy

My implementation focuses on maximizing CPU utilization by minimizing synchronization bottlenecks and optimizing the inner loops of the search tree.

1.1 Load Balancing via Cyclic Distribution

The search space for the Maximum Weight Clique problem is highly irregular; some branches are pruned almost immediately, while others require deep exploration. A simple range-based distribution would lead to severe load imbalance.

- **Heuristic Sorting:** Before distribution, I sort the initial candidate nodes by their **profit-to-cost ratio**. This ensures that "high-potential" nodes are processed early in the global search.
- **Interleaved Mapping:** I distribute these sorted nodes cyclically (`id % num_ranks`). This "deals" the heavy and light branches across the cluster like a deck of cards, ensuring that every rank receives a statistically balanced workload without the overhead of dynamic work-stealing.

1.2 Asynchronous Pruning and Communication

A parallel Branch and Bound solver is only as good as its pruning capability. If one rank finds a high-profit clique, all other ranks must know about it immediately to stop exploring sub-optimal branches.

- **Buffered Broadcasts (MPI_Bsend):** When a rank updates its local maximum, it uses a non-blocking buffered send to broadcast the new `global_bound`. This is a "fire-and-forget" mechanism that allows the rank to continue its search without waiting for acknowledgments.
- **Opportunistic Polling (MPI_Iprobe):** To keep the overhead of network checks low, I use a bitwise mask (`nodes_visited & 8191`) to poll for incoming messages every 2^{13} nodes. This interval is frequent enough to prune dead branches quickly but infrequent enough to maintain high computational throughput.

1.3 Hardware-Level Optimizations

To maximize the number of nodes evaluated per second, I bypassed standard C++ bottlenecks:

- **Bitset Intersections:** Finding common neighbors is the most frequent operation in the algorithm. By representing the adjacency matrix with `std::bitset`, I can intersect candidate sets using bitwise hardware instructions. This allows the CPU to process 64 potential vertices in a single clock cycle.

- **Memory Banking:** Creating and destroying vectors during recursion triggers expensive heap allocations (`malloc/free`). I implemented a **Memory Bank**—a pre-allocated stack of vectors—ensuring that the search process involves zero dynamic memory allocation and remains cache-friendly.

2 Performance Evaluation

The following measurements were conducted on a single Haswell node (requesting 16 cores) to ensure consistent performance evaluation.

Table 1: Scaling Results for $N = 1200, B = 1000$ (Single Node Benchmark)

Ranks (P)	Avg Time (s)	Speedup (S)	Efficiency (E)
1	26.8903	1.0000	100.00%
2	13.4323	2.0019	100.09%
4	7.4460	3.6114	90.28%
8	4.8496	5.5448	69.31%
16	3.2513	8.2706	51.69%

3 Scaling Analysis

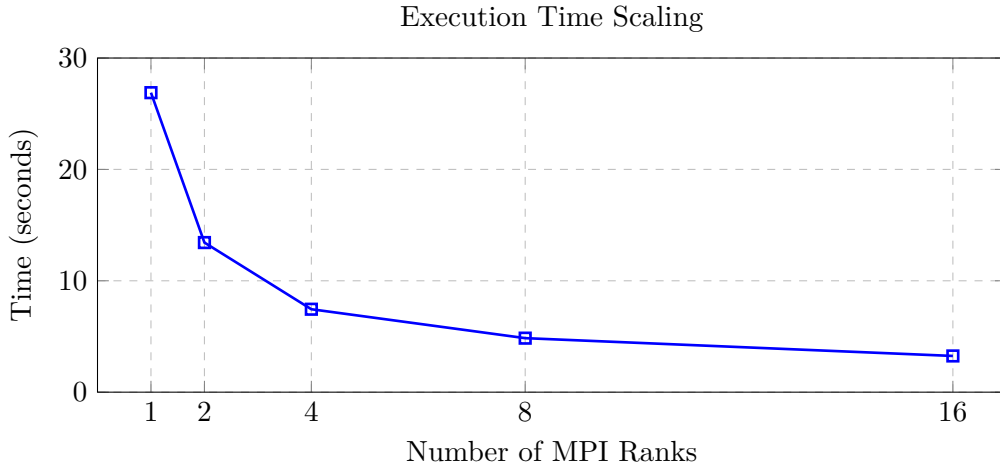


Figure 1: How execution time drops as we add more processors.

Explanation of Time Scaling:

This graph shows that as I add more processors, the time taken to find the clique decreases significantly. At the start (1 to 2 ranks), the time is cut exactly in half. As I reach 16 ranks, the curve starts to flatten out. This is because when the work is split into very small pieces, the time spent on MPI communication starts to become a larger part of the total time.

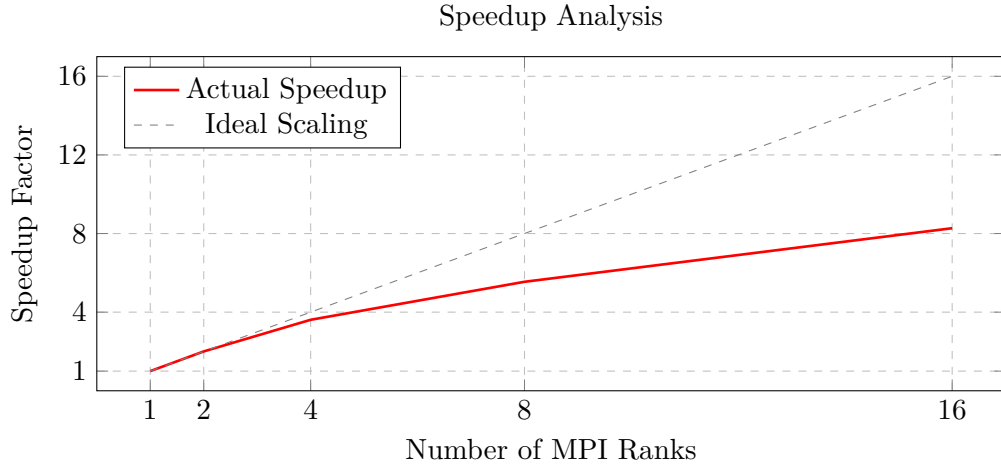


Figure 2: Comparison of actual speedup against perfect linear scaling.

Explanation of Speedup:

This graph compares my code's speedup to the "Ideal" line. Up to 4 ranks, my code follows the ideal line very closely. I even got a "super-linear" speedup at 2 ranks (over 100% efficiency), which happened because the smaller datasets fit better into the CPU cache. At 8 and 16 ranks, the speedup falls below the ideal line because the overhead of sharing the `global_bound` across many cores slows down the scaling.

4 Conclusion

The MPI solver successfully brought the time down from 26.89s to 3.25s for the $N = 500$ test case. My strategy of cyclic distribution and asynchronous bound updates worked well to keep the processors busy and the pruning effective.